

TITLE: - CONVENTION MEMORY

AIM: -

To write a program to find info of the Keyboard special keys state and display the Key state (On/Off)

ALGORITHM: -

- Initialize data segment.
- Display the header message.
- Execute BIOS INT 16h (Function 02h) to read keyboard status flags.
- Save the keyboard flags returned in the AL register.
- Test bit 6 and display Caps Lock status (ON/OFF).
- Test bit 5 and display Num Lock status (ON/OFF).
- Test bit 4 and display Scroll Lock status (ON/OFF).
- Test bit 7 and display Insert Key status (ON/OFF).
- Test bits 0 and 1 and display Shift Key status (ON/OFF).
- Test bit 2 and display Ctrl Key status (ON/OFF).
- Test bit 3 and display Alt Key status (ON/OFF).
- Terminate program execution.

PROGRAM:-

TITLE KeySp.ASM -- Display Keyboard Special Key Status

```
;/*****
;
;/*
;/* Copyright (c) 1985-2025, American Megatrends International LLC.  *
;/*
;/* All rights reserved. Subject to AMI licensing agreement.      *
;/*
;/***/
```

```
; <AMI_FHDR_START>
;
; Header:
; Author: Muthuganesh S
; Description:
; Program to read and display keyboard special key states
; like Caps Lock, Num Lock, Scroll Lock, Insert,
; Shift, Ctrl and Alt using BIOS keyboard status.
;
; <AMI_FHDR_END>
```

```
.model small
```

```
.386
```

```
.stack 100h
```

```
;-----
```

```
; DATA SEGMENT
```

```
;-----
```

```
data segment
```

```
msg1 db 13,10,"Keyboard Special Key Status",13,10,13,10,''
```

```
capsmsg db "Caps Lock : $"
```

```
nummsg db 13,10,"Num Lock : $"
```

```
scrollmsg db 13,10,"Scroll Lock : $"
```

```
insertmsg db 13,10,"Insert Key : $"
```

```
shiftmsg db 13,10,"Shift Key : $"
```

```
ctrlmsg db 13,10,"Ctrl Key : $"
```

```
altmsg db 13,10,"Alt Key : $"
```

```
onmsg db "ON$"
```

```
offmsg db "OFF$"
```

```
keystate db ?
```

```
data ends
```

```
;-----
```

```
;-----
```

```
; CODE SEGMENT
```

```
;-----
```

```
code segment
```

```
assume cs:code, ds:data
```

```
start:
```

```
mov ax, data
```

```
mov ds, ax
```

```
;-----
```

```
; Display Header
```

```
;-----
```

```
mov dx, offset msg1
```

```
mov ah, 09h
```

```
int 21h
```

```
;-----
```

```
; BIOS Keyboard Status
; INT 16h Function 02h
; Returns keyboard flags in AL
;-----
mov ah, 02h
int 16h

mov keystate, al

;-----
; Caps Lock Status (Bit 6)
;-----

mov dx, offset capsmsg
mov ah, 09h
int 21h

test keystate, 01000000b
jz caps_off

caps_on:
    mov dx, offset onmsg
    jmp caps_print

caps_off:
    mov dx, offset offmsg

caps_print:
    mov ah, 09h
    int 21h
```

```
;------  
; Num Lock Status (Bit 5)  
;------  
  
mov dx, offset nummsg  
mov ah, 09h  
int 21h  
  
test keystate, 00100000b  
jz num_off  
  
num_on:  
    mov dx, offset onmsg  
    jmp num_print  
  
num_off:  
    mov dx, offset offmsg  
  
num_print:  
    mov ah, 09h  
    int 21h  
  
;------  
; Scroll Lock Status (Bit 4)  
;------  
  
mov dx, offset scrollmsg  
mov ah, 09h  
int 21h
```

```
test keystate, 00010000b
```

```
jz scroll_off
```

```
scroll_on:
```

```
mov dx, offset onmsg
```

```
jmp scroll_print
```

```
scroll_off:
```

```
mov dx, offset offmsg
```

```
scroll_print:
```

```
mov ah, 09h
```

```
int 21h
```

```
;-----
```

```
; Insert Key Status (Bit 7)
```

```
;-----
```

```
mov dx, offset insertmsg
```

```
mov ah, 09h
```

```
int 21h
```

```
test keystate, 10000000b
```

```
jz insert_off
```

```
insert_on:
```

```
mov dx, offset onmsg
```

```
jmp insert_print
```

```
insert_off:
```

```
mov dx, offset offmsg
```

```
insert_print:
```

```
mov ah, 09h
```

```
int 21h
```

```
;-----
```

```
; Shift Key Status (Bits 0 and 1)
```

```
;-----
```

```
mov dx, offset shiftmsg
```

```
mov ah, 09h
```

```
int 21h
```

```
test keystate, 00000011b
```

```
jz shift_off
```

```
shift_on:
```

```
mov dx, offset onmsg
```

```
jmp shift_print
```

```
shift_off:
```

```
mov dx, offset offmsg
```

```
shift_print:
```

```
mov ah, 09h
```

```
int 21h
```

```
;-----
```

```
; Ctrl Key Status (Bit 2)
```

```
;-----
```

```
mov dx, offset ctrlmsg
mov ah, 09h
int 21h
test keystate, 00000100b
jz ctrl_off
```

ctrl_on:

```
mov dx, offset onmsg
jmp ctrl_print
```

ctrl_off:

```
mov dx, offset offmsg
```

ctrl_print:

```
mov ah, 09h
```

```
int 21h
```

```
;-----
```

```
; Alt Key Status (Bit 3)
```

```
;-----
```

```
mov dx, offset altmsg
```

```
mov ah, 09h
```

```
int 21h
```

```
test keystate, 00001000b
```

```
jz alt_off
```

alt_on:

```
mov dx, offset onmsg
```

```
jmp alt_print
```

alt_off:

```
mov dx, offset offmsg
```

```
alt_print:
```

```
mov ah, 09h
```

```
int 21h
```

```
;-----
```

```
; Exit Program
```

```
;-----
```

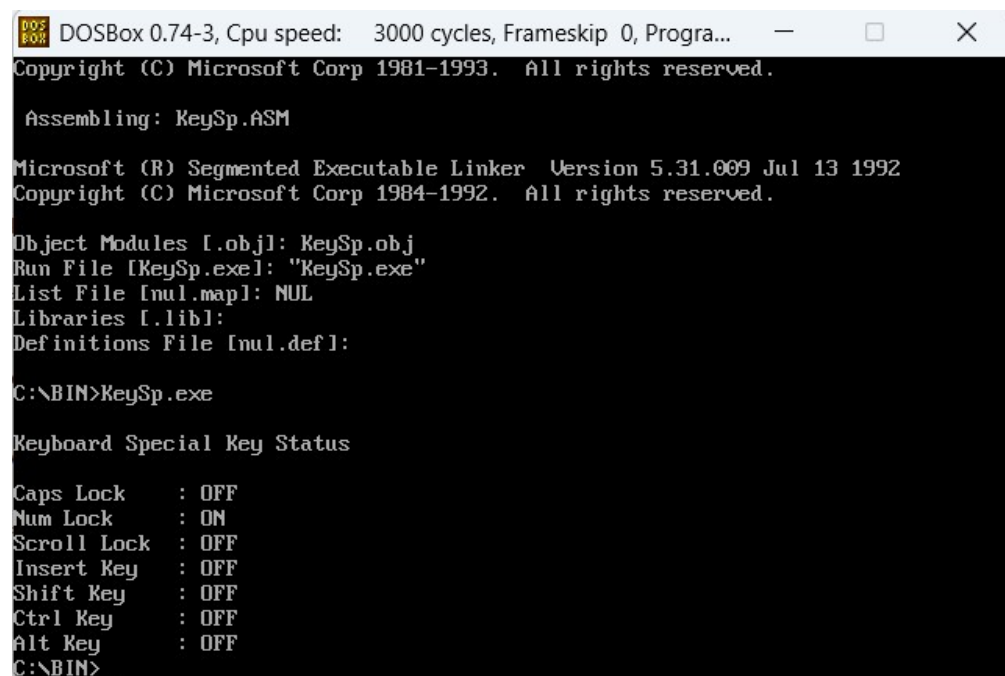
```
mov ah, 4Ch
```

```
int 21h
```

```
code ends
```

```
end start
```

OUTPUT: -



```
DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Progra...
Copyright (C) Microsoft Corp 1981-1993. All rights reserved.

Assembling: KeySp.ASM

Microsoft (R) Segmented Executable Linker Version 5.31.009 Jul 13 1992
Copyright (C) Microsoft Corp 1984-1992. All rights reserved.

Object Modules [.obj]: KeySp.obj
Run File [KeySp.exe]: "KeySp.exe"
List File [nul.map]: NUL
Libraries [.lib]:
Definitions File [nul.def]:

C:\BIN>KeySp.exe

Keyboard Special Key Status

Caps Lock      : OFF
Num Lock       : ON
Scroll Lock    : OFF
Insert Key     : OFF
Shift Key      : OFF
Ctrl Key       : OFF
Alt Key        : OFF
C:\BIN>
```

AIM: -

To display the starting address of EBDA and Size of the EBDA

ALGORITHM: -

- Initialize the data segment.
- Display the header message.
- Read the EBDA segment address from the BIOS Data Area at memory location 0040:000E.
- Call the subroutine to convert and display the segment address in hexadecimal.
- Calculate the 32-bit physical address by shifting the segment address left by 4 bits (multiplying by 16).
- Call the subroutine to convert and display the physical address in hexadecimal.
- Read the first byte of the EBDA segment to retrieve its size in Kilobytes.
- Display the EBDA size in both hexadecimal and formatted string outputs.
- Terminate program execution.

PROGRAM:-

TITLE EBDA.ASM -- Display EBDA Start Address and Size

```
;/******  
;  
;/*  
;/* Copyright (c) 1985-2025, American Megatrends International LLC. *  
;  
;/*  
;/* All rights reserved. Subject to AMI licensing agreement. *  
;  
;/******  
  
;<AMI_FHDR_START>  
;  
; Header:  
; Author: Muthuganesh S
```

; Description:

; Program to display the starting address and size

; of Extended BIOS Data Area (EBDA).

;

; <AMI_FHDR_END>

.model small

.386

.stack 100h

;-

; DATA SEGMENT

;-

data segment

msg1 db 13,10,"EBDA Information",13,10,13,10,'\$'

msg2 db "EBDA Segment Address : 0x\$"

msg3 db 13,10,"EBDA Physical Address: 0x\$"

msg4 db 13,10,"EBDA Size : 0x\$"

hexbuf db '0000\$'

hexbuf32 db '00000000\$'

ebda_seg dw ?

ebda_size db ?

data ends

;-

```
;-----  
;   CODE SEGMENT  
;-----  
  
code segment  
assume cs:code, ds:data  
  
start:  
  
    mov ax, data  
    mov ds, ax  
  
;-----  
; Print Header  
;-----  
  
    mov dx, offset msg1  
    mov ah, 09h  
    int 21h  
  
;-----  
; EBDA Segment Pointer  
; BIOS Data Area  
; 0040:000E  
;-----  
  
    mov ax, 0040h  
    mov es, ax  
  
    mov ax, es:[000Eh]
```

```
mov ebda_seg, ax
```

```
;-----
```

```
; Print EBDA Segment Address
```

```
;-----
```

```
mov dx, offset msg2
```

```
mov ah, 09h
```

```
int 21h
```

```
mov ax, ebda_seg
```

```
call PrintHexWord
```

```
;-----
```

```
; Calculate Physical Address
```

```
; Physical = Segment * 16
```

```
;-----
```

```
movzx eax, ebda_seg
```

```
shl eax, 4
```

```
;-----
```

```
; Print Physical Address
```

```
;-----
```

```
mov dx, offset msg3
```

```
mov ah, 09h
```

```
int 21h
```

```
call PrintHexDword
```

```
;-----
```

```
; Read EBDA Size
; First byte of EBDA contains size in KB
;-----
mov ax, ebda_seg
mov es, ax

mov al, es:[0000h]
mov ebda_size, al

;-----
; Print EBDA Size
;-----

mov dx, offset msg4
mov ah, 09h
int 21h

xor ah, ah
mov al, ebda_size

call PrintHexWord

mov dl, 'h'
mov ah, 02h
int 21h

mov dl, ''
int 21h

mov dl, '('
```

int 21h

mov al, ebda_size

add al, '0'

mov dl, al

int 21h

mov dl, 'K'

int 21h

mov dl, 'B'

int 21h

mov dl, ')'

int 21h

;------

; Exit Program

;------

mov ah, 4Ch

int 21h

;------

; PrintHexWord Procedure

;------

PrintHexWord proc

push ax

push bx

push cx

push dx

push si

mov si, offset hexbuf

mov cx, 4

hex_loop1:

rol ax, 4

mov bl, al

and bl, 0Fh

cmp bl, 9

jbe digit1

add bl, 7

digit1:

add bl, '0'

mov [si], bl

inc si

loop hex_loop1

```
mov dx, offset hexbuf
```

```
mov ah, 09h
```

```
int 21h
```

```
pop si
```

```
pop dx
```

```
pop cx
```

```
pop bx
```

```
pop ax
```

```
ret
```

```
PrintHexWord endp
```

```
;-----
```

```
; PrintHexDword Procedure
```

```
;-----
```

```
PrintHexDword proc
```

```
push eax
```

```
push ebx
```

```
push ecx
```

```
push edx
```

```
push si
```

```
mov si, offset hexbuf32
```

```
mov cx, 8
```

hex_loop2:

rol eax, 4

mov bl, al

and bl, 0Fh

cmp bl, 9

jbe digit2

add bl, 7

digit2:

add bl, '0'

mov [si], bl

inc si

loop hex_loop2

mov dx, offset hexbuf32

mov ah, 09h

int 21h

pop si

pop edx

pop ecx

pop ebx

pop eax

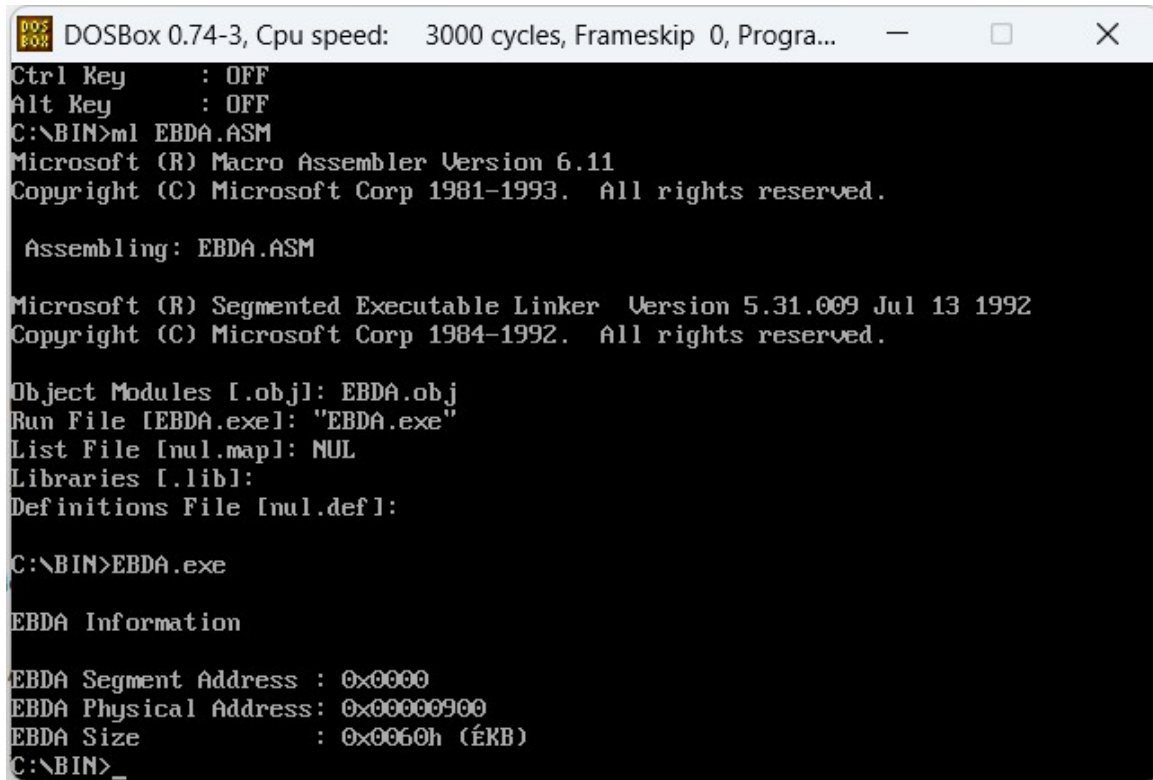
ret

PrintHexDword endp

code ends

end start

OUTPUT: -



```
DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Progra...
Ctrl Key      : OFF
Alt Key       : OFF
C:\BIN>ml EBDa.ASM
Microsoft (R) Macro Assembler Version 6.11
Copyright (C) Microsoft Corp 1981-1993. All rights reserved.

Assembling: EBDa.ASM

Microsoft (R) Segmented Executable Linker Version 5.31.009 Jul 13 1992
Copyright (C) Microsoft Corp 1984-1992. All rights reserved.

Object Modules [.obj]: EBDa.obj
Run File [EBDa.exe]: "EBDa.exe"
List File [nul.map]: NUL
Libraries [.lib]:
Definitions File [nul.def]:

C:\BIN>EBDa.exe

EBDa Information

EBDa Segment Address : 0x0000
EBDa Physical Address: 0x00000900
EBDa Size             : 0x0060h (6KB)
C:\BIN>_
```

AIM: -

To write a program to verify the presence and no. of Option ROM's available in the system and transfer the program control to VGA OpROM and regain the control back in the program.

ALGORITHM: -

- Initialize the data segment.
- Display the scanning start message.
- Set the initial memory segment pointer to C000h.
- Check the first two bytes of the current segment for the valid Option ROM signature (55h and AAh).
- If a valid signature is found, increment the ROM counter and display the detected segment address.
- If the detected ROM is at segment C000h (VGA ROM), execute a far call to C000:0003 and regain control afterward.
- Read the ROM size byte at offset 2 and convert it into a segment increment value.
- Add the increment value to the current segment pointer to jump to the next potential ROM boundary.
- Repeat the scanning loop until the segment pointer reaches F000h.
- Display the total count of discovered Option ROMs.
- Terminate program execution.

PROGRAM:-

TITLE OPROM.ASM -- Detect Option ROM and Transfer Control to VGA ROM

```
;//*****  
;  
;  
;//* Copyright (c) 1985-2025, American Megatrends International LLC. *  
;  
;  
;//* All rights reserved. Subject to AMI licensing agreement. *  
;  
;  
;//*****  
  
;<AMI_FHDR_START>  
;
```

```
; Header:
; Author: Muthuganesh S
; Description:
; Program to scan Option ROM area, detect available
; Option ROMs, count them and transfer control to
; VGA Option ROM and regain control back.
```

```
;
;<AMI_FHDR_END>
```

```
.model small
.386
.stack 100h
```

```
;-----
;   DATA SEGMENT
;-----
data segment
```

```
msg1 db 13,10,"Scanning Option ROM Area...",13,10,' '
msg2 db 13,10,"Option ROM Found at Segment : $"
msg3 db 13,10,"Total Option ROM Count   : $"
msg4 db 13,10,"Calling VGA Option ROM...",13,10,' '
msg5 db 13,10,"Returned From VGA Option ROM",13,10,' '
```

```
hexbuf db '0000$'
countbuf db '00$'
```

```
romcount db 0
```

data ends

;-----

;-----

; CODE SEGMENT

;-----

code segment

assume cs:code, ds:data

start:

mov ax,data

mov ds,ax

;-----

; Print Start Message

;-----

mov dx, offset msg1

mov ah,09h

int 21h

;-----

; Start Scan from C000h

;-----

mov ax,0C000h

mov es,ax

scan_loop:

```
;-----  
; Check ROM Signature 55AAh  
;-----  
mov bx,0  
  
mov al, es:[bx]  
cmp al,55h  
jne next_segment  
  
mov al, es:[bx+1]  
cmp al,0AAh  
jne next_segment  
  
;-----  
; Valid ROM Found  
;-----  
inc romcount  
  
mov dx, offset msg2  
mov ah,09h  
int 21h  
  
mov ax, es  
call PrintHexWord  
  
;-----  
; VGA ROM Usually at C000h  
;-----
```

```
mov ax, es
cmp ax,0C000h
jne skip_vga
```

```
mov dx, offset msg4
mov ah,09h
int 21h
```

```
;-----
; Transfer Control to VGA ROM
; Entry Point = C000:0003
;-----
pushf
```

```
db 09Ah
dw 0003h
dw 0C000h
```

```
mov dx, offset msg5
mov ah,09h
int 21h
```

```
skip_vga:
```

```
next_segment:
```

```
;-----
; Get ROM Size
; Offset 2 = Size in 512-byte blocks
```

```
;-----  
mov al, es:[0002h]  
  
cmp al,0  
jne valid_size  
  
mov al,1  
  
valid_size:  
  
xor ah,ah  
  
;-----  
; Convert to Segment Increment  
; 512 bytes = 20h paragraphs  
;-----  
mov bx,20h  
mul bx  
  
;-----  
; ES = ES + Size  
;-----  
mov bx,es  
add bx,ax  
mov es,bx  
  
;-----  
; Continue Until F000h  
;-----
```

```
mov bx,es
cmp bx,0F000h
jb scan_loop
```

```
;-----
```

```
; Print Total ROM Count
```

```
;-----
```

```
mov dx, offset msg3
```

```
mov ah,09h
```

```
int 21h
```

```
mov al, romcount
```

```
call PrintHexByte
```

```
;-----
```

```
; Exit Program
```

```
;-----
```

```
mov ah,4Ch
```

```
int 21h
```

```
;-----
```

```
; PrintHexWord Procedure
```

```
;-----
```

```
PrintHexWord proc
```

```
push ax
```

```
push bx
```

```
push cx
```

push dx

push si

mov si, offset hexbuf

mov cx,4

hex_loop1:

rol ax,4

mov bl,al

and bl,0Fh

cmp bl,9

jbe digit1

add bl,7

digit1:

add bl,'0'

mov [si],bl

inc si

loop hex_loop1

mov dx, offset hexbuf

mov ah,09h

int 21h

pop si

pop dx

pop cx

pop bx

pop ax

ret

PrintHexWord endp

;------

; PrintHexByte Procedure

;------

PrintHexByte proc

push ax

push bx

push dx

mov bl,al

shr al,4

call HexAscii

mov countbuf[0],al

mov al,bl

```
and al,0Fh
```

```
call HexAscii
```

```
mov countbuf[1],al
```

```
mov dx, offset countbuf
```

```
mov ah,09h
```

```
int 21h
```

```
pop dx
```

```
pop bx
```

```
pop ax
```

```
ret
```

```
PrintHexByte endp
```

```
;-----
```

```
; HexAscii Procedure
```

```
;-----
```

```
HexAscii proc
```

```
    cmp al,9
```

```
    jbe digit2
```

```
    add al,7
```

```
digit2:
```

```
add al,'0'
```

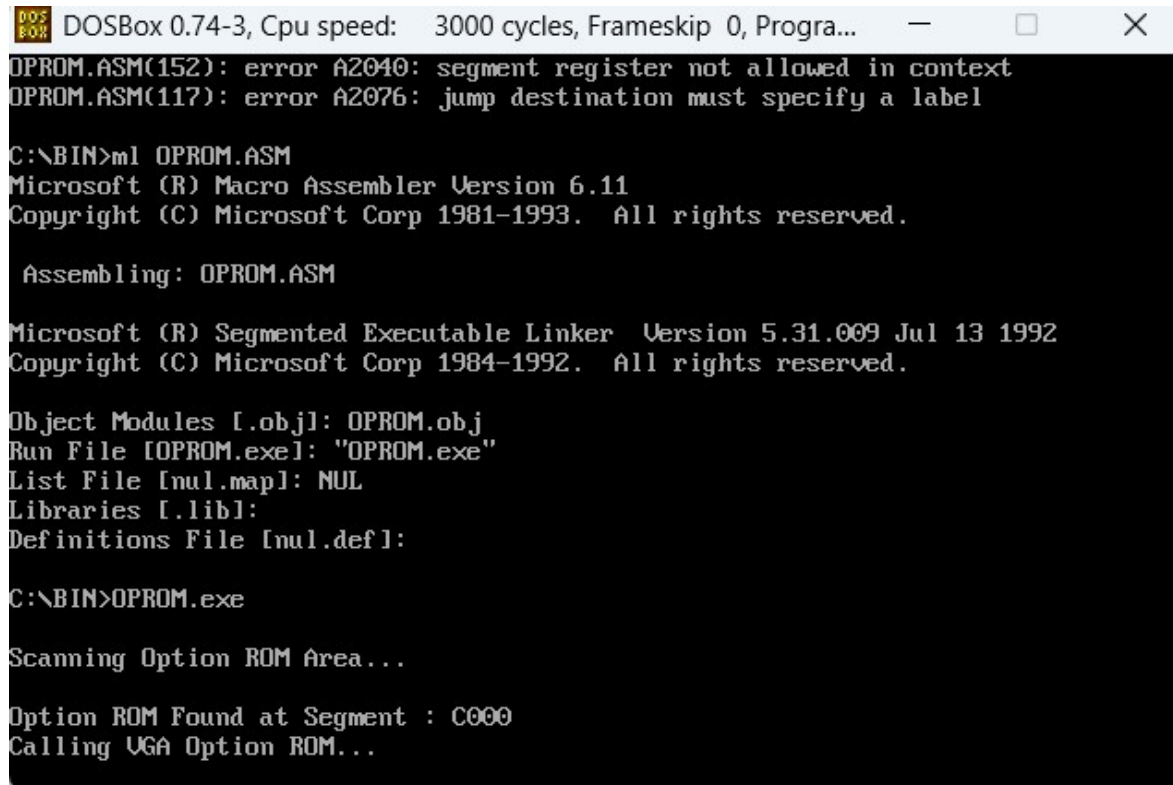
```
ret
```

```
HexAscii endp
```

```
code ends
```

```
end start
```

OUTPUT: -



```
DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Progra...
OPROM.ASM(152): error A2040: segment register not allowed in context
OPROM.ASM(117): error A2076: jump destination must specify a label

C:\BIN>ml OPROM.ASM
Microsoft (R) Macro Assembler Version 6.11
Copyright (C) Microsoft Corp 1981-1993. All rights reserved.

Assembling: OPROM.ASM

Microsoft (R) Segmented Executable Linker Version 5.31.009 Jul 13 1992
Copyright (C) Microsoft Corp 1984-1992. All rights reserved.

Object Modules [.obj]: OPROM.obj
Run File [OPROM.exe]: "OPROM.exe"
List File [nul.map]: NUL
Libraries [.lib]:
Definitions File [nul.def]:

C:\BIN>OPROM.exe

Scanning Option ROM Area...

Option ROM Found at Segment : C000
Calling UGA Option ROM...
```